CHAPTER 2

METHODOLOGY

2.1 BLOCK DIAGRAM

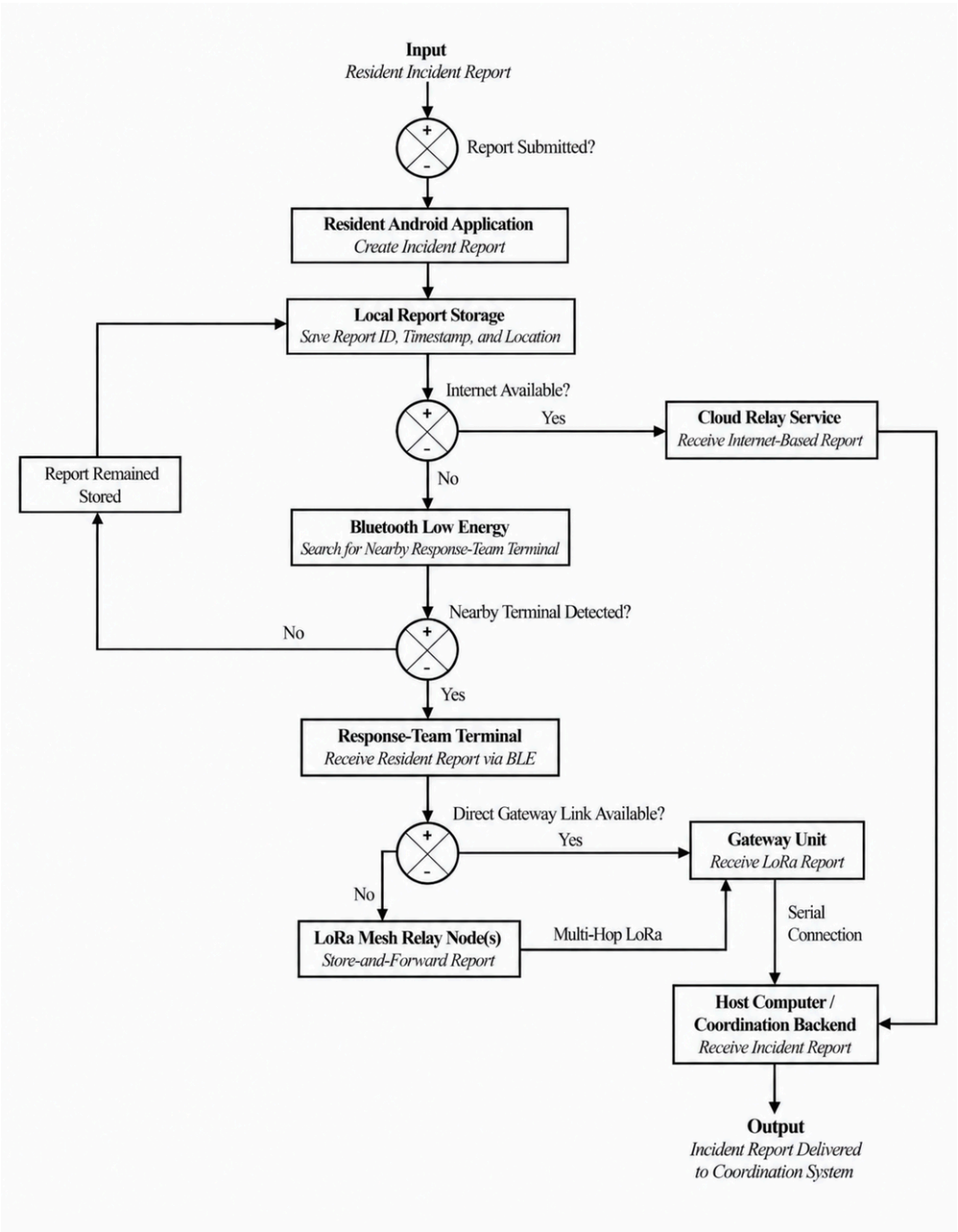


Figure 2.1.1 Resident Reporting and LoRa Mesh Communication Block Diagram

The resident reporting and LoRa mesh communication subsystem provided the communication path through which incident reports submitted by residents were delivered to the SALBAR coordination backend under both connected and disconnected network conditions. The process began when a resident created an incident report through the Android application. Upon submission, the report was first saved in the device's local report storage together with its report identifier, timestamp, and location. This local storage served as the persistent copy of the report before transmission and allowed the application to retain the information when an immediate communication path was unavailable. This behavior followed the offline-first reporting architecture of SALBAR, in which reports were preserved locally rather than discarded when internet connectivity was lost.

After the report had been stored, the application determined whether internet connectivity was available. When an internet connection was present, the report was transmitted to the cloud relay service, from which the host computer and coordination backend retrieved the submitted incident data. This path allowed the resident application to deliver reports without involving the LoRa gateway when commercial network infrastructure remained accessible. When internet connectivity was unavailable, the application instead used Bluetooth Low Energy to search for a nearby response-team terminal. If no terminal was detected within Bluetooth range, the report remained stored locally and was retained for a later relay opportunity. Once a nearby terminal became available, the stored report was transferred through BLE to the response-team terminal for forwarding through the infrastructure-independent communication network.

After receiving the resident report through BLE, the response-team terminal attempted to forward the message toward the gateway unit. When a direct LoRa connection to the gateway was available, the terminal transmitted the report directly. Otherwise, the report was passed

through one or more LoRa mesh relay nodes using store-and-forward communication until a viable path to the gateway became available. This mechanism allowed intermediate terminals to retain messages during temporary communication interruptions and forward them once connectivity was restored. Upon receiving the LoRa report, the gateway unit transferred the incident data to the host computer through a serial connection. Both the internet-based cloud path and the offline BLE-LoRa path ultimately delivered the resident incident report to the coordination backend for subsequent processing by the SALBAR system.

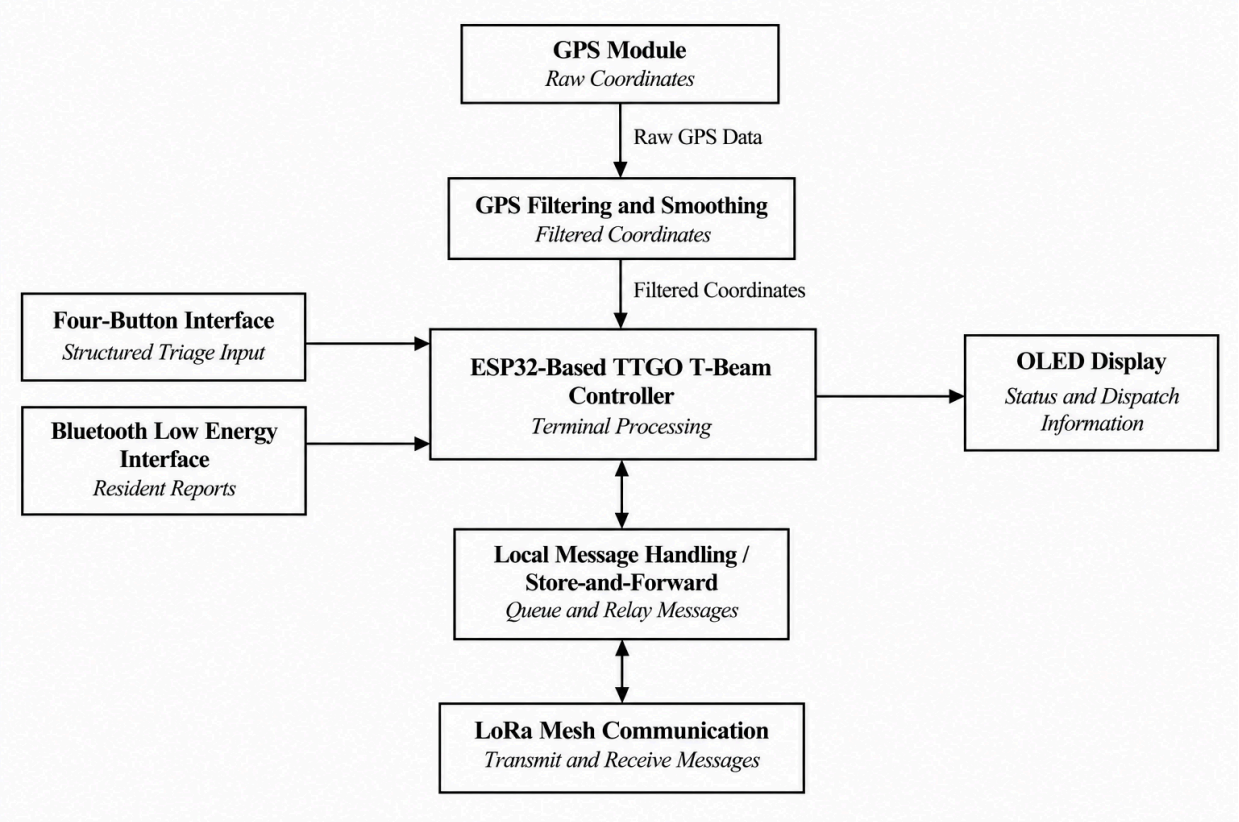


Figure 2.1.2 Response-Team Field Terminal Block Diagram

The response-team field terminal subsystem provided the hardware and communication interface used by deployed responders to submit structured field information, receive resident

reports, exchange messages through the LoRa mesh, and receive approved dispatch instructions. The ESP32-based TTGO T-Beam controller served as the central processing component of the terminal and coordinated the connected input, output, positioning, and communication functions. The terminal integrated GPS-based location tagging, a four-button interface for structured triage reporting, Bluetooth Low Energy communication, an OLED display, and LoRa mesh connectivity as defined in the SALBAR system architecture.

The GPS module supplied raw coordinate data to the GPS filtering and smoothing stage before the resulting coordinates were passed to the TTGO T-Beam controller. This intermediate processing reduced fluctuations and inaccuracies in the raw GPS measurements before the location information was used by the terminal. The controller also received structured responder input through the four-button interface, which supported field triage reporting, while the Bluetooth Low Energy interface allowed resident incident reports to be received from nearby Android devices when internet connectivity was unavailable. The processed terminal status and received dispatch information were presented to the responder through the OLED display.

The controller exchanged messages with the local message handling and store-and-forward function, which managed messages that required transmission or relay through the LoRa network. When an immediate communication path was unavailable, messages could be retained locally until a viable forwarding path became available. The LoRa mesh communication block then provided bidirectional transmission between response-team terminals and the gateway, allowing the field terminal to send incident reports, responder location and status information, and relayed messages while also receiving approved dispatch instructions from the coordination system.

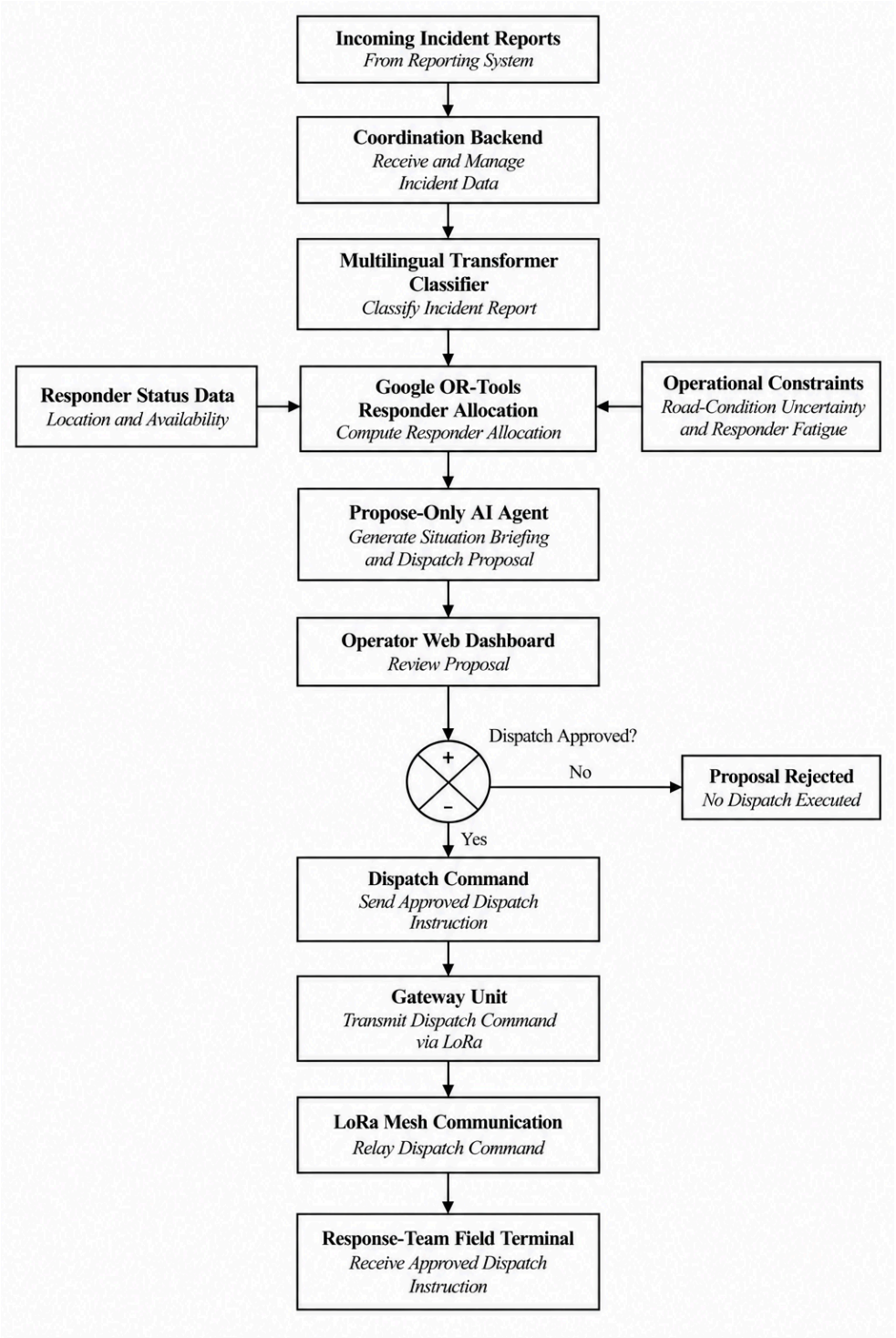


Figure 2.1.3 Coordination Backend and Human-in-the-Loop Dispatch Block Diagram

The coordination backend and human-in-the-loop dispatch subsystem processed incoming incident reports and supported the selection and dispatch of available response teams. Incident reports delivered by the reporting system were first received and managed by the coordination backend before being passed to the multilingual transformer classifier. The classifier processed reports written in English, Tagalog, Cebuano, and supported code-switched combinations to determine the incident information required by the coordination process. The classified report was then supplied to the Google OR-Tools responder allocation component for assignment computation.

The Google OR-Tools component computed responder allocations using the classified incident information together with responder status data and operational constraints. Responder status data provided information such as responder location and availability, while road-condition uncertainty and responder fatigue were incorporated as constraints affecting the allocation process. The resulting responder allocation was passed to the propose-only AI agent, which generated a situation briefing and dispatch proposal for presentation through the operator web dashboard. The AI agent remained limited to a recommendation role and did not independently authorize or execute responder deployment.

Through the operator web dashboard, the human operator reviewed the generated proposal and determined whether the recommended dispatch would be approved. When the proposal was rejected, no dispatch command was executed. When approval was granted, the system generated the corresponding dispatch command and forwarded the approved instruction to the gateway unit. The gateway transmitted the command through the LoRa mesh communication network until it reached the appropriate response-team field terminal. This

approval stage maintained the human-in-the-loop requirement of SALBAR by ensuring that responder dispatch occurred only after explicit operator authorization.

2.2 PROJECT MANAGEMENTspe

2.2.1 Materials and Resources

Table 2.1 detailed the physical components and resources utilized in the development and assembly of the SALBAR system prototype. The LilyGO TTGO T-Beam mainboards, featuring integrated GPS and LoRa transceivers, served as the primary processing units for the three handheld response-team terminals. A single LilyGO TTGO LoRa32 mainboard functioned as the central gateway unit to aggregate incoming telemetry packets. To construct the structured triage interface, 0.96-inch OLED displays and tactile push buttons were integrated into the field terminals using universal prototype PCBs and silicone wiring. Omnidirectional whip antennas and a high-gain base station antenna were installed to maximize the penetration and range of the LoRa transmissions through physical obstructions. The portable units were powered by 18650 Li-ion rechargeable batteries to ensure continuous operation independent of commercial electrical grids. Finally, to protect the circuits during field deployment, the field terminals were housed in custom 3D-printed cases, and the gateway was secured inside an IP65 weatherproof enclosure before connecting to the host computer via a serial data cable.

Table 2.1 Materials and Resources

Material/Resource	Quantity	Cost
LilyGO TTGO T-Beam V1.2 (ESP32, LoRa 915 MHz, GNSS/GPS)	3 units	₱7,050

LilyGO TTGO LoRa32 V2.1 (ESP32, LoRa 915 MHz, OLED)	1 unit	₱1,750
0.96-inch (or 1.3-inch) I2C OLED Display Module	3 units	₱420
915 MHz 3–5 dBi SMA Male Omnidirectional Whip Antenna	3 units	₱660
915 MHz 5–8 dBi High-Gain Base Station Antenna	1 unit	₱380
18650 Li-ion 3.7V Cells (2600–3200 mAh)	4 units	₱880
Tactile Push Button Switches (6x6mm or 12x12mm)	1 pack	₱120
Universal FR4 Dotted Double-Sided Prototype PCB	5 pcs	₱175
Resistor & Capacitor Assortment	1 pack	₱80
40-pin Dupont Jumpers + 28 AWG Ribbon Wire	1 lot	₱180
2.54mm Breakaway Male & Female Pin Headers	1 pack	₱60

Shielded USB-A to USB-C (or Micro-USB) Data Cable	1 pc	₱150
IP65 Weatherproof ABS Plastic Project Box	1 unit	₱250
3D-Printed Custom Handheld Cases	3 units	₱1,200
M2 / M2.5 / M3 Nylon Hex Spacers & Screws	1 kit	₱200
Dual-Slot External 18650 Battery Smart Charger	1 unit	₱250
63/37 Rosin Core Solder Wire, Flux, & Shrink Tubing	1 lot	₱200
Shipping Fees & Spares	1 lot	₱600
Total Cost		

2.2.2 Gantt Chart

Table 2.2 Gantt Chart

Task	Sept				Oct				Nov				Dec				Jan				Feb				March				April			
	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
Project Planning																																
Proposal Submission																																
Proposal Approval																																
Chapter 1																																
Chapter 2																																
Chapter 3																																
Increment 1																																
Increment 2																																
Increment 3																																
Increment 4																																
Chapter 4																																
References																																
Appendices																																

2.3 PROTOTYPE DESIGN

Make a short description of the prototype of your design. Describe each component of the prototype and state the reason why the component is designed that way.

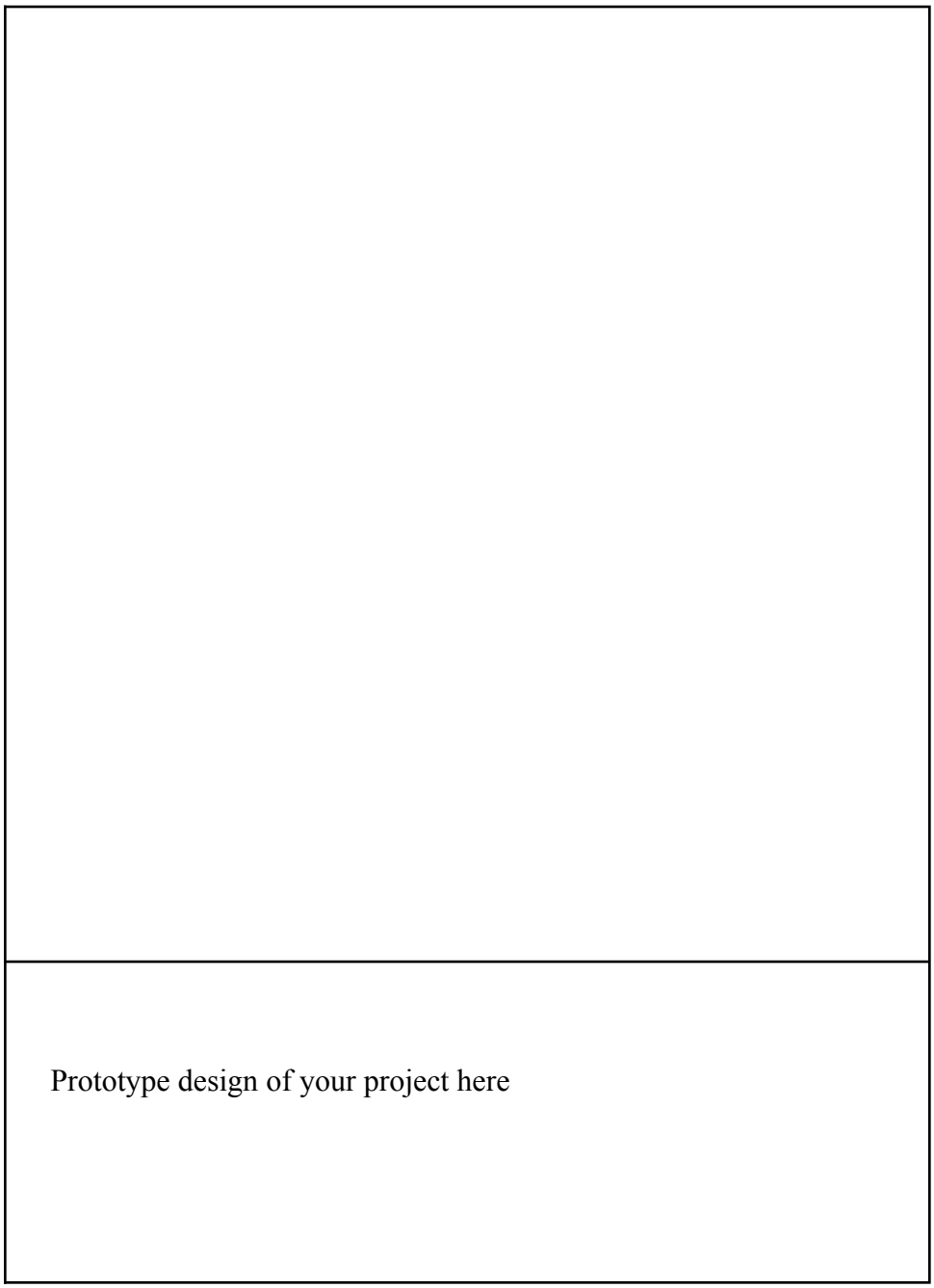


Figure 2.2 Prototype Design

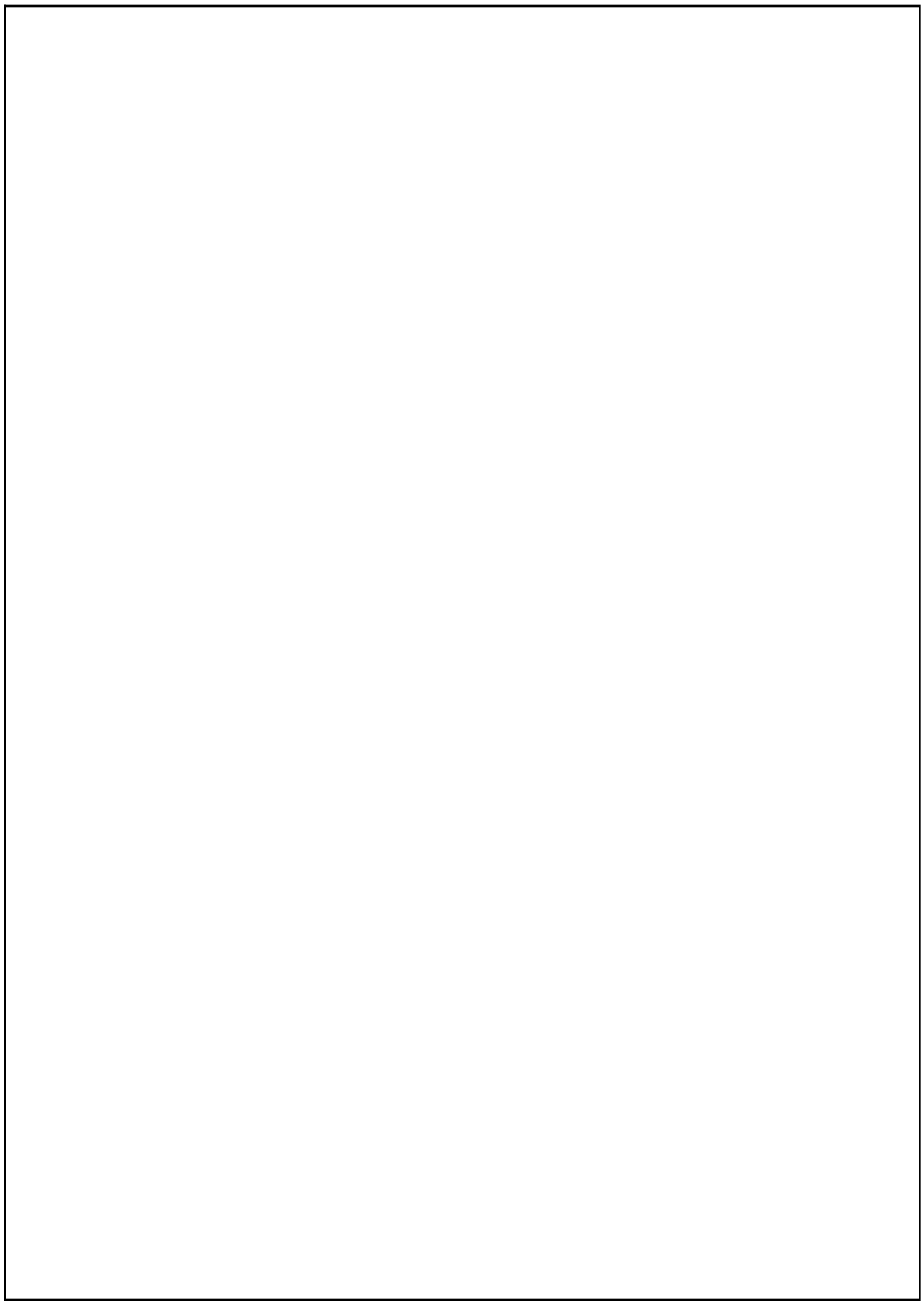
2.4 CIRCUIT DESIGN

Describe the purpose of the subsystem1 and its purpose in the project.

Circuit Design here, if possible break down by subsystem for clarity of discussion.

Figure 2.3 Subsystem1 Circuit Design

Describe the purpose of the subsystem2 and its purpose in the project.



Circuit Design here, if possible break down by subsystem for clarity of discussion.

Figure 2.4 Subsystem2 Circuit Design

Add more figures and discussion when necessary.

2.5 PROGRAM FLOWCHART

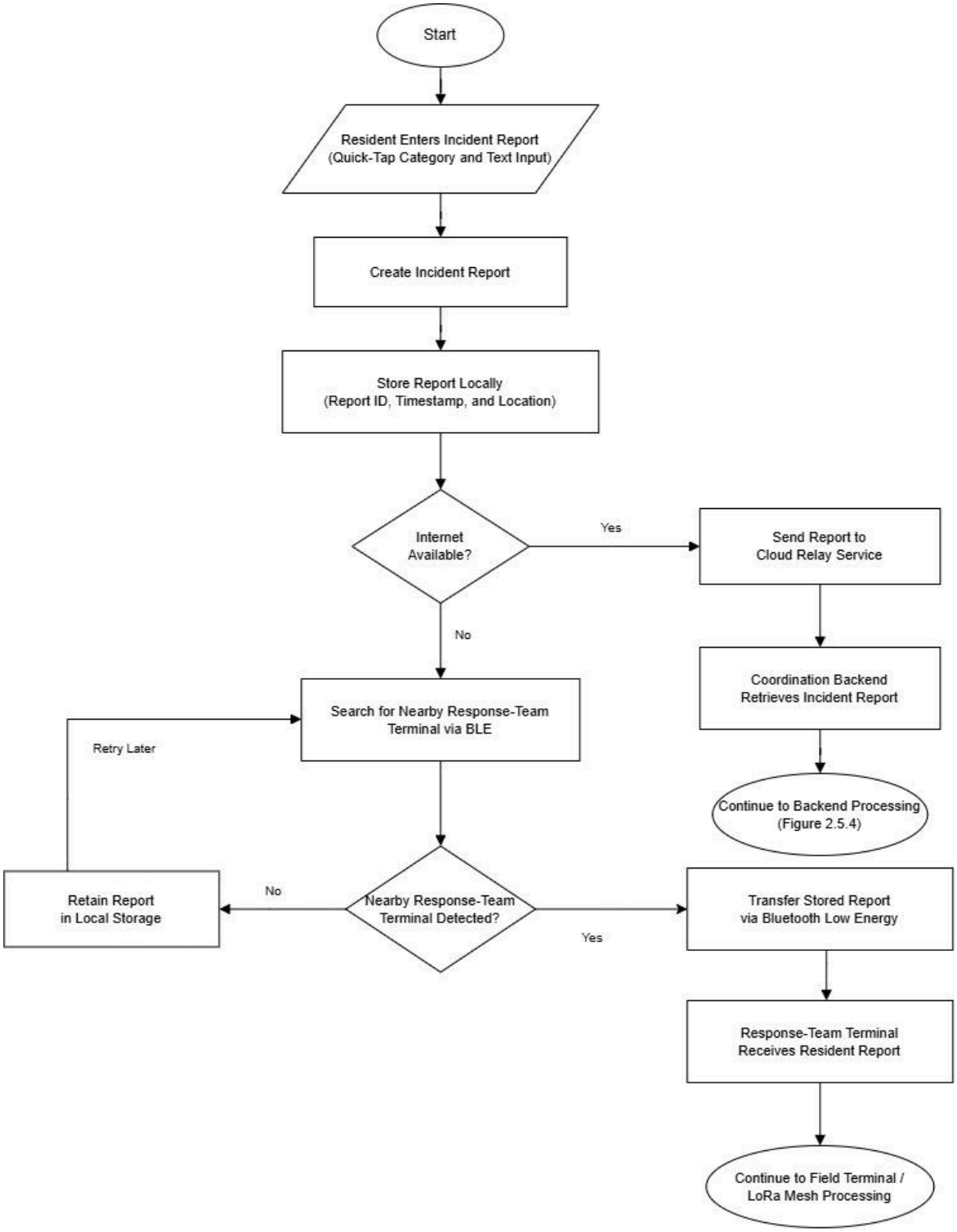


Figure 2.5.1 Resident Incident Reporting Program Flowchart

The resident incident reporting program flowchart illustrated the process followed by the SALBAR Android application when a resident submitted an emergency report. The process began when the resident entered the incident information using the application's quick-tap emergency category and text input. Upon submission, the application created the incident report and stored it locally together with its report identifier, timestamp, and location. Maintaining a local copy ensured that the report remained available even when an immediate communication path could not be established.

After storing the report, the application determined whether internet connectivity was available. When an internet connection was present, the report was transmitted to the cloud relay service, from which the coordination backend retrieved the submitted incident data for subsequent processing. When internet connectivity was unavailable, the application instead searched for a nearby response-team terminal using Bluetooth Low Energy. If a terminal was detected, the stored report was transferred through BLE to the response-team terminal for forwarding through the SALBAR communication network. If no terminal was within Bluetooth range, the report remained in local storage and the application retained it until another relay opportunity became available. This process implemented SALBAR's offline-first reporting approach by providing both an internet-based transmission path and an alternative BLE-assisted path during communication outages.

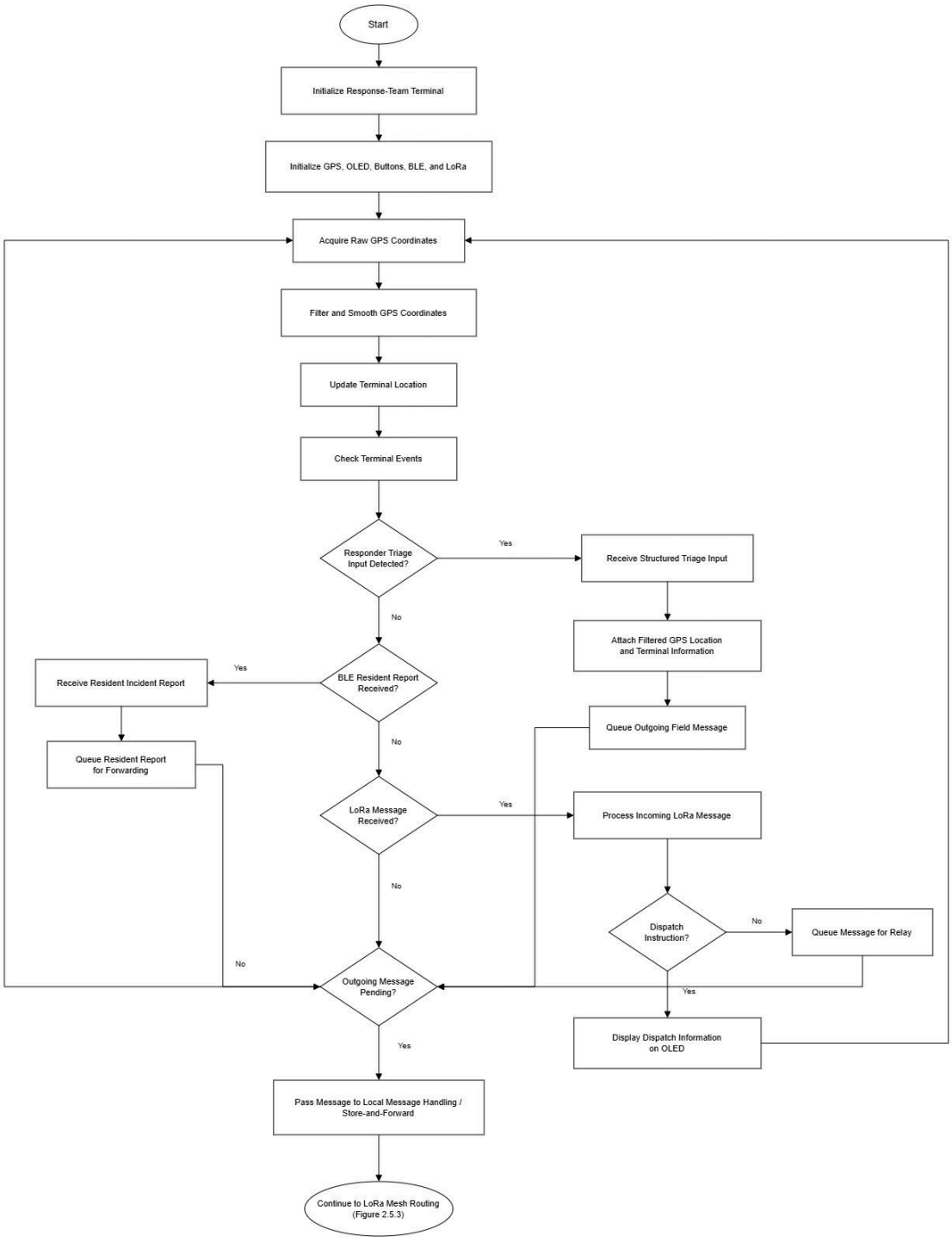


Figure 2.5.2 Response-Team Field Terminal Program Flowchart

The response-team field terminal program flowchart illustrated the operating sequence of the ESP32-based TTGO T-Beam terminal used by deployed responders. The process began with the initialization of the terminal and its GPS, OLED display, four-button interface, Bluetooth Low Energy interface, and LoRa communication functions. Once initialized, the terminal acquired raw GPS coordinates, processed them through the filtering and smoothing stage, and updated its current location before checking for terminal events. Returning to GPS acquisition during subsequent operating cycles allowed the terminal to refresh its location information while supporting responder input and communication functions.

The terminal first determined whether structured triage input had been detected through the four-button interface. When input was present, it received the entered information, attached the filtered GPS location and terminal information, and queued the resulting field message for transmission. When no triage input was detected, the terminal checked whether a resident incident report had been received through BLE. A received resident report was accepted and queued for forwarding. If no BLE report was received, the terminal checked for an incoming LoRa message. Received LoRa messages were processed to determine whether they contained an approved dispatch instruction addressed to the receiving terminal. Instructions intended for that terminal were displayed on the OLED, after which the terminal returned to GPS acquisition. Other received messages requiring forwarding, including dispatch instructions intended for another terminal, were queued for relay.

Following the report-queuing or relay branches, or when no incoming LoRa message was detected, the terminal determined whether an outgoing message was pending. When a pending message was available, it was passed to the local message-handling and store-and-forward mechanism before proceeding to the LoRa mesh routing process described in Figure 2.5.3. When

no outgoing message was pending, the terminal returned to GPS acquisition and continued its operating cycle. This sequence supported structured responder reporting, BLE-assisted resident-report forwarding, location updating, mesh message relaying, and the reception of approved dispatch instructions intended for the terminal.



Figure 2.5.3 LoRa Mesh Store-and-Forward Routing Flowchart

The LoRa mesh store-and-forward routing flowchart illustrated the proposed message-forwarding procedure performed by a SALBAR mesh node. The process began when a pending message was supplied to the routing function. The node retained the message locally and determined its destination before evaluating the available communication path. Field reports and responder telemetry were directed toward the gateway, while approved dispatch instructions were directed toward the intended response-team terminal. When a direct LoRa link to the destination was available, the node attempted to transfer the message directly. When no direct link was available, it checked for a suitable relay node and attempted to forward the message through that node.

Following either transfer attempt, the procedure checked whether the transfer had been confirmed by the receiving node. A confirmed transfer marked the forwarding operation of the current node as complete. When the receiving node was an intermediate relay, this confirmation represented completion of the current forwarding hop rather than delivery to the final destination; the receiving relay subsequently continued message handling and routing toward that destination. If the transfer was not confirmed, or if no suitable relay node was available, the message remained stored locally as pending for a later retry. This retention supported store-and-forward communication during temporary interruptions in mesh connectivity.

Both the completed-forwarding and pending-retry branches returned control to the calling program. For a response-team field terminal, control returned to GPS acquisition in Figure 2.5.2, allowing location updating and terminal-event processing to continue while undelivered messages remained available for subsequent routing attempts. During a later attempt, the node reconsidered direct destination connectivity before checking for a relay opportunity. This procedure supported the forwarding of field information toward the gateway and the return of

human-approved dispatch instructions toward the appropriate field terminals without requiring cellular or internet connectivity.

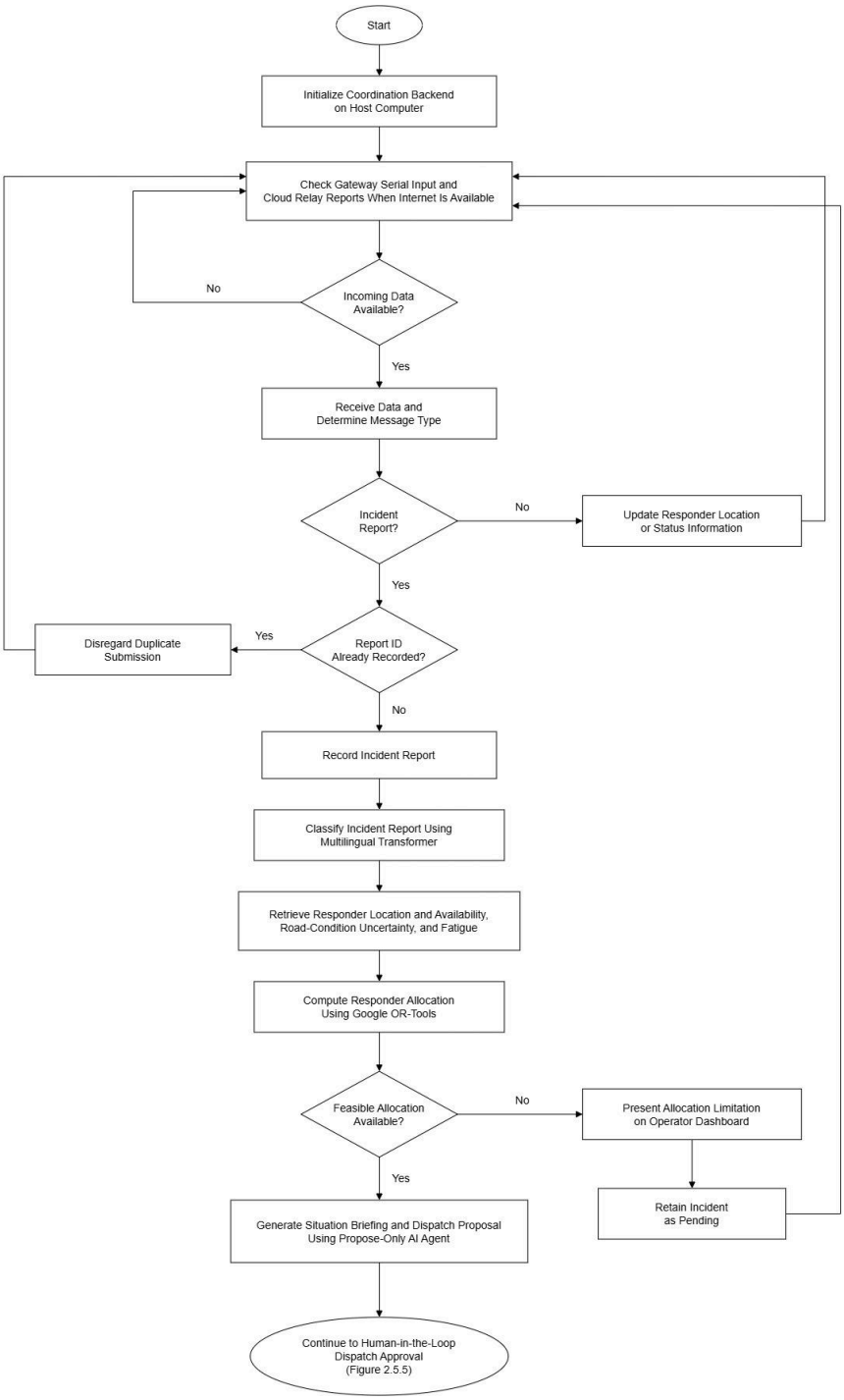


Figure 2.5.4 Coordination Backend and Responder Allocation Program Flowchart

The coordination backend and responder allocation program flowchart illustrated the processing sequence performed on the SALBAR host computer. The process began with backend initialization, followed by monitoring incoming data through the gateway's serial connection and checking cloud relay reports when host internet connectivity was available. When no incoming data was available, the backend continued monitoring these communication paths. Upon receiving data, it determined whether the message contained an incident report or responder location and status information. Responder updates were incorporated into the backend's status information before monitoring resumed. Incident reports were checked against previously recorded report identifiers. Duplicate submissions were disregarded, while new reports were recorded for subsequent processing.

Each newly recorded incident report was processed by the multilingual transformer classifier, which supported English, Tagalog, Cebuano, and code-switched text. The backend then retrieved responder location and availability information together with operational constraints concerning road-condition uncertainty and responder fatigue. These inputs, along with the classified incident information, were used by Google OR-Tools to compute a responder allocation. The process subsequently determined whether a feasible allocation was available. When no feasible allocation was produced, the allocation limitation was presented through the operator dashboard, the incident remained pending, and the backend returned to incoming-data monitoring.

When a feasible allocation was available, the resulting assignment information was passed to the propose-only AI agent to generate a situation briefing and dispatch proposal. The proposal then proceeded to the human-in-the-loop dispatch approval process described in Figure 2.5.5. This handoff maintained the distinction between computing a recommended allocation and

authorizing responder deployment. The classifier, allocation engine, and AI agent operated on the host computer, and no dispatch command was executed through this processing sequence without subsequent human approval.

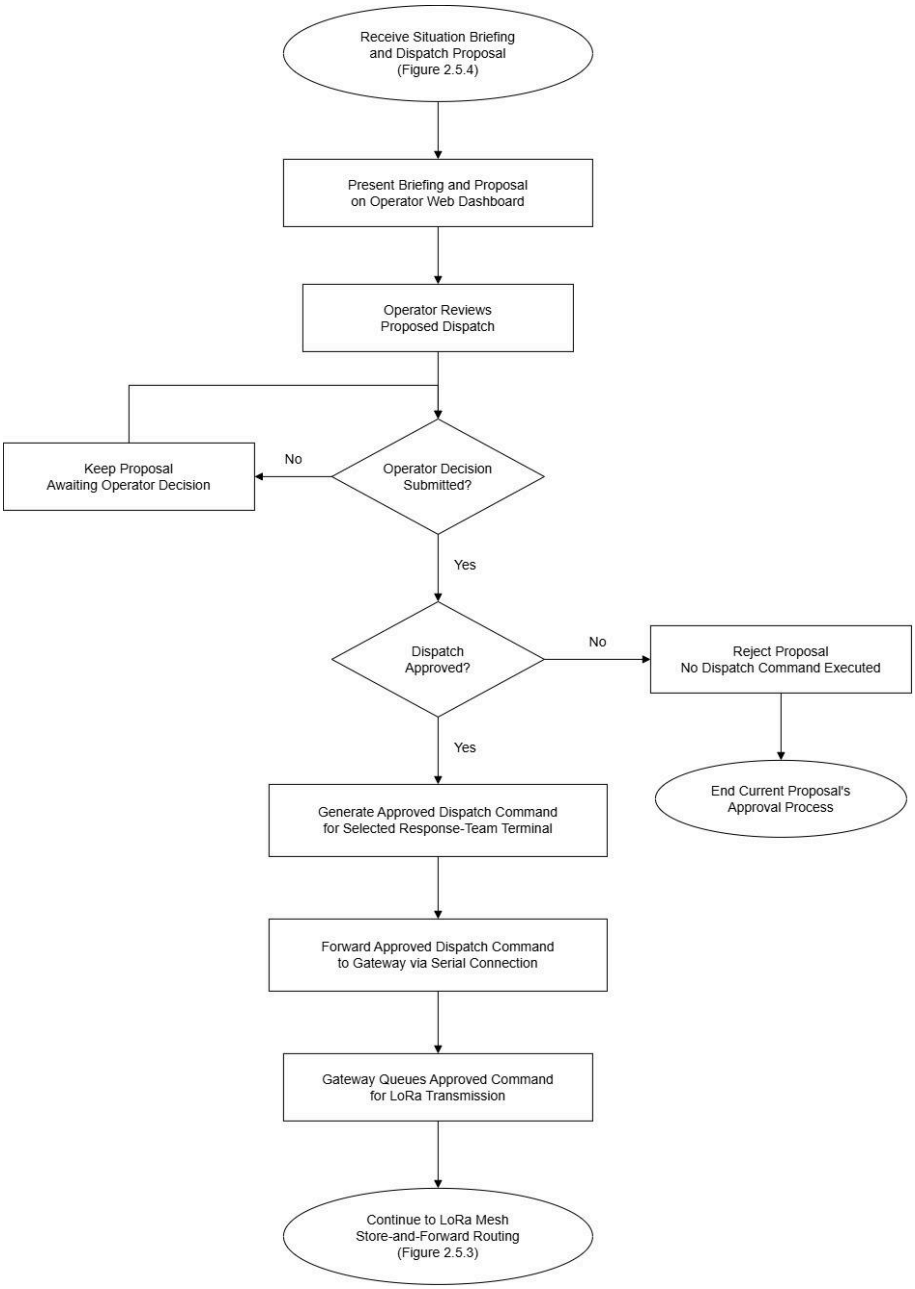


Figure 2.5.5 Human-in-the-Loop Dispatch Approval Program Flowchart

The human-in-the-loop dispatch approval program flowchart illustrated the operator authorization process applied to the situation briefing and dispatch proposal generated in Figure 2.5.4. The briefing and proposal were presented through the operator web dashboard for human review. The process then checked whether the operator had submitted a decision. When no decision had been submitted, the proposal remained awaiting review, and the process continued checking for an explicit operator response. This waiting state did not constitute rejection or authorize command generation.

Once an operator decision was submitted, the process determined whether the proposed dispatch had been approved. A rejected proposal resulted in no dispatch command being executed and concluded the approval process for that proposal. When approval was granted, the coordination backend generated the corresponding dispatch command for the selected response-team terminal. The approved command was forwarded to the gateway through the serial connection, where it was queued for LoRa transmission before continuing to the store-and-forward routing procedure described in Figure 2.5.3.

This approval sequence maintained human authority over responder dispatch by requiring explicit authorization before command generation and transmission. The AI agent remained limited to producing briefings and proposals and could not independently execute dispatch. Subsequent mesh forwarding and reception by the intended terminal were handled by the routing and field-terminal procedures, respectively.

2.6 TESTING AND DATA GATHERING

2.6.1 Testing Strategy 1: Evaluating Mesh Communication Reliability

Description:

The study evaluated the transmission reliability of the infrastructure-independent LoRa mesh network by measuring key performance indicators such as packet delivery rate (PDR), latency, and maximum transmission range. The tests were conducted using the ESP32-based TTGO T-Beam terminals and the central gateway, specifically focusing on scenarios where direct line-of-sight was deliberately blocked to trigger the network's store-and-forward relay capabilities.

Data Gathered:

(To be filled after testing)

Importance of Data:

Gathering these metrics was critical to determine the network's capacity to sustain emergency telemetry and routing commands during simulated commercial cellular and internet outages.

Purpose of Testing:

The results were intended to prove the robustness of the decentralized mesh topology, demonstrating that disaster incident reports are not permanently dropped during temporary network partitions, but are reliably bounced from device to device until reaching the command center.

2.6.2 Testing Strategy 2: Determining GPS Accuracy and Filtering Efficacy

Description:

The study tested the accuracy of the onboard GPS location tagging and the mathematical efficacy of the signal filtering and smoothing algorithms embedded within the handheld response terminals. Measurements were taken in diverse environmental conditions, including simulated dense canopies and obstructed urban areas, to assess how well the firmware mitigated coordinate scattering and multipath fading.

Data Gathered:

(To be filled after testing)

Importance of Data:

Highly precise and stable location tagging was an absolute prerequisite for the coordination backend, as the operations-research engine relied entirely on this telemetry to accurately map responder positions and compute viable detour routes.

Purpose of Testing:

This testing aimed to prove that raw, noisy GPS data could be successfully corrected at the hardware edge prior to LoRa transmission, ensuring that the command center received reliable coordinates rather than erratic positional drift.

2.6.3 Testing Strategy 3: Assessing Multilingual Classification Accuracy

Description:

The study evaluated the fine-tuned transformer-based NLP model's capability to correctly interpret and categorize unstructured disaster reports written in English, Tagalog, Cebuano, and highly code-switched combinations. The classifier was tested against a self-assembled disaster dataset to measure its precision in extracting accurate emergency categories and urgency levels.

Data Gathered:

(To be filled after testing)

Importance of Data:

This data was necessary to validate the AI layer's ability to process the informal, panic-driven text inputs submitted by residents through the mobile app without treating mixed-language phrases as mere spelling errors.

Purpose of Testing:

The results sought to prove that the coordination backend could actively and reliably triage unstructured civilian reports into structured, actionable situation briefings for the incident commander.

2.6.4 Testing Strategy 4: Evaluating Responder Allocation Performance

Description:

The study compared the automated dispatch efficiency of the Google OR-Tools responder allocation engine against a standard, non-optimized first-come, first-served (FCFS) operational baseline. The evaluation explicitly incorporated dynamic deployment constraints, calculating how the routing algorithm penalized specific paths based on simulated road-condition blockages and accumulated responder fatigue.

Data Gathered:

(To be filled after testing)

Importance of Data:

Gathering these performance metrics was vital to measure the real-world utility of transitioning from a retrospective, manual dispatch model to a proactive, algorithmic allocation system.

Purpose of Testing:

This testing aimed to prove that utilizing constraint programming and stochastic optimization significantly reduced logistical inefficiencies, ensuring responders were assigned tasks based on proximity, stamina, and actual route viability rather than mere chronological report order.

2.6.5 Testing Strategy 5: Determining Overall System Usability and Integration

Description:

The study conducted simulated field stress trials and timed usability drills across both the hardware and software endpoints. This involved timing the submission of structured triage reports via the four-button OLED interface of the response terminals, as well as testing the Bluetooth (BLE) opportunistic data transfer between the offline resident Android application and a passing responder's hardware terminal.

Data Gathered:

(To be filled after testing)

Importance of Data:

Usability data ensured that the system's interfaces were highly accessible and intuitive, confirming that neither panicked civilians nor burdened frontline responders were hindered by complex data-entry requirements during a crisis.

Purpose of Testing:

The testing was designed to demonstrate the seamless end-to-end integration of the SALBAR architecture, proving that the offline-first mobile app, the hardware nodes, and the Human-in-the-Loop backend successfully closed the communication and dispatch loop.